

Краткое сравнение библиотек для написания конкурентных программ

Поддержка конкурентности и многопоточности в языках программирования и библиотеках не нова, несмотря на сравнительно недавнюю стандартизацию такой поддержки в C++. Например, в языке Java поддержка многопоточности была уже в его первых выпусках. На платформах, соответствующих стандарту POSIX, предоставляется интерфейс языка C, предназначенный для многопоточности. А в языке Erlang предоставляется поддержка для реализации конкурентности на основе отправки сообщений. Существуют даже библиотеки классов C++, например Boost, в которые заключается соответствующий интерфейс программирования многопоточности, используемый в любой отдельно взятой платформе (будь то интерфейс POSIX C или что-либо иное), предназначенные для того, чтобы предоставить интерфейс, переносимый между поддерживаемыми платформами.

Для тех, у кого уже есть опыт написания многопоточных приложений и желание использовать новые свойства многопоточности C++, в этом приложении предлагаю сравнение свойств, доступных в Java, POSIX C, C++ с Boost Thread Library и C++11. Приведу также перекрестные ссылки на соответствующие главы в этой книге.

Функция	Java	POSIX C	Boost Threads	C++11	Ссылка на главу
Запуск потоков	Класс <code>java.lang.Thread</code>	Тип <code>pthread_t</code> и связанные с ним API-функции: <code>pthread_create()</code> , <code>pthread_detach()</code> и <code>pthread_join()</code>	Класс <code>boost::thread</code> и компонентные функции	Класс <code>std::thread</code> и компонентные функции	Глава 2
Взаимное исключение	Блоки <code>synchronized</code>	Тип <code>pthread_mutex_t</code> и связанные с ним API-функции: <code>pthread_mutex_lock()</code> , <code>pthread_mutex_unlock()</code> и т. д.	Класс <code>boost::mutex</code> и компонентные функции, а также шаблоны <code>boost::lock_guard<></code> и <code>boost::unique_lock<></code>	Класс <code>std::mutex</code> и компонентные функции, а также шаблоны <code>std::lock_guard<></code> и <code>std::unique_lock<></code>	Глава 3
Отслеживание и ожидание предиката	Методы <code>wait()</code> и <code>notify()</code> класса <code>java.lang.Object</code> , используемые внутри блоков <code>synchronized</code>	Тип <code>pthread_cond_t</code> и связанные с ним API-функции: <code>pthread_cond_wait()</code> , <code>pthread_cond_timed_wait()</code> и т. д.	Классы <code>boost::condition_variable</code> и <code>boost::condition_variable_any</code> и компонентные функции	Классы <code>std::condition_variable</code> и <code>std::condition_variable_any</code> и компонентные функции	Глава 4
Атомарные операции и модель памяти с принципом конкурентности	<code>volatile</code> -переменные, типы в пакете <code>java.util.concurrent.atomic</code>	Отсутствует	Отсутствует	Типы <code>std::atomic_XXX</code> , шаблон класса <code>std::atomic<></code> , функция <code>std::atomic_thread_fence()</code>	Глава 5
Потокобезопасные контейнеры	Контейнеры в пакете <code>java.util.concurrent</code>	Отсутствует	Отсутствует	Отсутствует	Главы 6 и 7
Фьючерсы	Интерфейс <code>java.util.concurrent.Future</code> и связанные с ними классы	Отсутствует	Шаблоны классов <code>boost::unique_future<></code> и <code>boost::shared_future<></code>	Шаблоны классов <code>std::future<></code> , <code>std::shared_future<></code> и <code>std::atomic_future<></code>	Глава 4
Пулы потоков	Класс <code>java.util.concurrent.ThreadPoolExecutor</code>	Отсутствует	Отсутствует	Отсутствует	Глава 9
Прерывание потока	Метод <code>interrupt()</code> класса <code>java.lang.Thread</code>	<code>pthread_cancel()</code>	Компонентная функция <code>interrupt()</code> класса <code>boost::thread</code>	Отсутствует	Глава 9